

1. 顺序容器

vector (动态数组)

特点:
适用场景:
基本使用方法:

deque (双端队列)

特点:
适用场景:
基本使用方法:

list (双向链表)

特点:
适用场景:
基本使用方法:

forward_list (单向链表)

特点:
适用场景:
基本使用方法:

2. 关联容器

set (有序集合)

特点:
适用场景:
基本使用方法:

multiset (多重集合)

特点:
适用场景:
基本使用方法:

map (键值对映射)

特点:
适用场景:
基本使用方法:

multimap (多重映射)

特点:
适用场景:
基本使用方法:

3. 无序关联容器

unordered_set (无序集合)

特点:
适用场景:
基本使用方法:

unordered_multiset (无序多重集合)

特点:
适用场景:
基本使用方法:

unordered_map (无序映射)

特点:
适用场景:
基本使用方法:

unordered_multimap (无序多重映射)

特点:
适用场景:
基本使用方法:

4. 容器适配器

stack (栈)

特点:
适用场景:

基本使用方法：

queue (队列)

特点：

适用场景：

priority_queue (优先队列)

特点：

适用场景：

5.选择建议

1. 顺序容器

vector (动态数组)

特点：

- 动态大小的数组，支持随机访问
- 尾部插入/删除效率高（摊销 $O(1)$ ）
- 中间插入/删除效率低（ $O(n)$ ）
- 内存连续存储

适用场景：

- 需要频繁随机访问元素
- 数据量动态变化
- 对缓存友好性有要求

基本使用方法：

```
#include <vector>
#include <iostream>

std::vector<int> vec = {1, 2, 3, 4, 5};
vec.push_back(6);           // 尾部插入
vec[0] = 10;                // 随机访问
std::cout << vec.size();    // 获取大小
vec.erase(vec.begin());    // 删除第一个元素
```

deque (双端队列)

特点：

- 支持两端高效插入/删除（ $O(1)$ ）
- 支持随机访问
- 内存分段连续

适用场景：

- 需要在两端进行插入/删除操作
- 需要随机访问
- 队列或栈的实现

基本使用方法：

```
#include <deque>

std::deque<int> deq = {1, 2, 3};
deq.push_front(0);      // 前端插入
deq.push_back(4);       // 后端插入
deq.pop_front();        // 前端删除
deq.pop_back();         // 后端删除
```

list（双向链表）

特点：

- 双向链表结构
- 任意位置插入/删除效率高（O(1)）
- 不支持随机访问
- 迭代器稳定性好

适用场景：

- 频繁在任意位置插入/删除
- 不需要随机访问
- 需要稳定的迭代器

基本使用方法：

```
#include <list>

std::list<int> lst = {1, 2, 3};
lst.push_front(0);
lst.push_back(4);
auto it = lst.begin();
++it;
lst.insert(it, 99);      // 在指定位置插入
lst.remove(2);           // 删除值为2的元素
```

forward_list（单向链表）

特点：

- 单向链表，内存开销小
- 只支持向前遍历
- 任意位置插入/删除效率高

适用场景：

- 内存受限的情况
- 只需要单向遍历
- 不需要反向操作

基本使用方法：

```
#include <forward_list>

std::forward_list<int> flst = {1, 2, 3};
flst.push_front(0);      // 只能在前端插入
flst.insert_after(flst.before_begin(), 99); // 在开始位置插入
```

2. 关联容器

set（有序集合）

特点：

- 存储唯一元素，自动排序
- 基于红黑树实现
- 插入/删除/查找效率为 $O(\log n)$

适用场景：

- 需要去重的集合
- 需要有序存储
- 快速查找操作

基本使用方法：

```
#include <set>

std::set<int> st = {3, 1, 4, 1, 5}; // 会自动去重并排序
st.insert(2);
st.erase(3);
if (st.count(1)) {                  // 查找元素是否存在
    std::cout << "Found 1\n";
}
```

multiset（多重集合）

特点：

- 允许重复元素的有序集合
- 基于红黑树实现
- 插入/删除/查找效率为 $O(\log n)$

适用场景：

- 需要维护有序序列但允许重复
- 统计元素出现次数

基本使用方法：

```
#include <set>

std::multiset<int> mst = {3, 1, 4, 1, 5};
mst.insert(1);          // 允许重复插入
auto range = mst.equal_range(1); // 获取值为1的所有元素范围
```

map（键值对映射）

特点：

- 存储键值对，键唯一，自动按键排序
- 基于红黑树实现
- 插入/删除/查找效率为 $O(\log n)$

适用场景：

- 键值对映射关系
- 需要按键排序
- 快速查找特定键对应的值

基本使用方法：

```
#include <map>
#include <string>

std::map<std::string, int> mp;
mp["apple"] = 5;
mp["banana"] = 3;
std::cout << mp["apple"];    // 访问值
if (mp.find("orange") != mp.end()) {    // 查找键
std::cout << "Found orange\n";
} else {
std::cout << "Orange not found\n";
}
```

multimap（多重映射）

特点：

- 允许重复键的键值对映射
- 基于红黑树实现
- 插入/删除/查找效率为 $O(\log n)$

适用场景：

- 一个键对应多个值的情况
- 需要维护键值对的有序性

基本使用方法：

```
#include <map>

std::multimap<int, std::string> mmap;
mmap.insert({1, "first"});
mmap.insert({1, "another"});    // 允许重复键
auto range = mmap.equal_range(1);    // 获取键为1的所有元素
```

3. 无序关联容器

unordered_set (无序集合)

特点:

- 基于哈希表实现
- 不维护元素顺序
- 平均插入/删除/查找效率为 $O(1)$

适用场景:

- 不需要元素有序
- 频繁的查找操作
- 性能要求较高的去重场景

基本使用方法:

```
#include <unordered_set>

std::unordered_set<int> ust = {3, 1, 4, 1, 5};
ust.insert(2);
if (ust.count(3)) {           // 检查元素是否存在
    std::cout << "3 exists\n";
}
ust.erase(3);
```

unordered_multiset (无序多重集合)

特点:

- 允许重复元素的无序集合
- 基于哈希表实现
- 平均操作效率为 $O(1)$

适用场景:

- 不需要有序但允许重复
- 高性能的统计场景

基本使用方法:

```
#include <unordered_set>

std::unordered_multiset<int> umst = {3, 1, 4, 1, 5};
umst.insert(1);           // 允许重复
size_t count = umst.count(1); // 统计元素出现次数
```

unordered_map (无序映射)

特点:

- 基于哈希表的键值对映射
- 不维护键的顺序
- 平均操作效率为 $O(1)$

适用场景：

- 不需要按键排序
- 高性能的键值查找
- 缓存实现

基本使用方法：

```
#include <unordered_map>

std::unordered_map<std::string, int> ump;
ump["hello"] = 1;
ump["world"] = 2;
if (ump.find("hello") != ump.end()) {
    std::cout << "value: " << ump["hello"] << std::endl;
}
```

unordered_multimap（无序多重映射）

特点：

- 允许重复键的无序键值对映射
- 基于哈希表实现
- 平均操作效率为 $O(1)$

适用场景：

- 一对多映射且不需要排序
- 高性能的多值查找

基本使用方法：

```
#include <unordered_map>

std::unordered_multimap<int, std::string> ummap;
ummap.insert({1, "first"});
ummap.insert({1, "second"});    // 重复键
auto range = ummap.equal_range(1); // 获取键为1的所有值
```

4. 容器适配器

stack（栈）

特点：

- LIFO（后进先出）结构
- 只能从顶部插入/删除
- 基于其他容器（通常是 deque）实现

适用场景：

- 撤销操作
- 表达式求值
- 递归算法的非递归实现

基本使用方法：

```
#include <stack>

std::stack<int> stk;
stk.push(1);
stk.push(2);
int top = stk.top();           // 获取栈顶元素
stk.pop();                     // 弹出栈顶元素
bool empty = stk.empty();      // 检查是否为空
```

queue（队列）

特点：

- FIFO（先进先出）结构
- 从队尾插入，队头删除
- 基于 deque 实现

适用场景：

- 广度优先搜索
- 任务调度
- 缓冲区实现
- **基本使用方法：**

```
#include <queue>

std::queue<int> q;
q.push(1);
q.push(2);
int front = q.front();        // 获取队首元素
q.pop();                       // 弹出队首元素
bool empty = q.empty();       // 检查是否为空
```

priority_queue（优先队列）

特点：

- 最大堆结构（默认）
- 总是返回最大元素
- 基于 vector 实现

适用场景：

- Dijkstra 算法
- 堆排序
- 任务优先级调度
- **基本使用方法：**


```
#include <queue>

std::priority_queue<int> pq;
pq.push(3);
pq.push(1);
pq.push(4);
int max_val = pq.top();    // 获取最大元素
pq.pop();                  // 弹出最大元素
```

5.选择建议

场景	推荐容器
随机访问，动态大小	vector
两端操作频繁	deque
中间插入/删除频繁	list
键值对映射，需要排序	map
键值对映射，追求性能	unordered_map
去重集合，需要排序	set
去重集合，追求性能	unordered_set
LIFO (后入先出)结构	stack
FIFO(先入先出) 结构	queue